CASE STUDY 25: Progressive Water Tariff Analysis

1, Problem Statement

Calculate water bills for several households using consumption slabs, a fixed fee, and an extra penalty for very high usage. Compare

usage and flag excessive consumption.

2. Objectives

+ Calculate the bill using progressive slabs.

+ Add a fixed fee and a high-usage penalty.

+ Find the household with the highest usage and flag excessive usage.

3. Assumptions

+ Rates are IBS per unit for the first 10 units, M8 per unit for the next 10, and 12 per unit above 20.

« Fixed fee is M30 per household.

+ Consumption above 30 units gets a M50 penalty.

+ All consumption values are non-negative.

4. Algorithm

1. Read the number of households.

2. For each household, read its name and water units.

3. Calculate slab charges, add the fixed fee, and apply the penalty if needed.

4. Display each bill and mark usage above 30 units as excessive.

5. Flowchart (text inside standard symbols)

6. Pseudocode ee

```
 START
INPUT number_of_households
 FOR each household:
INPUT name, units
 IF unlts <= 10: charge = units * $
 ELSE IF units <= 20: charge = 1045 + (units 10)+8
 ELSE: charge = 10°S + 10°8 + (units-20)°12
 bill = charge + 30
 IF units > 30: bill - bill + 50
 DISPLAY name, units, bill
 END FOR
 STOP
```

7. Python Source Code

```
n= int (input ("Number of households: ")}
 highest_oame = **
 bighest_onits = -1
 for i in range(n):
name = input ("Household name: *}
units = int (input ("Water units used: *)}
if units < 0:
print ("Invalid units")
continue
if units ¢= Lo:
 charge = units * 5
 elif units <= 20:
 charge = 10 + $ + (units - 10) * 8
 else:
 charge = 10+ 5 +104 8+ (units - 20) + 12
 bill = charge + 30
if units > 30:
Bill ¢= 50
```

Python Case Studies 2 34 B.Tech Ist Year Page 1

print ("Excessive usage!)

printinane, "bill = Rs.*, bill)

if units > bighest_units:

bighest units = units

```
bighest_name = name
```

if highest_units >= 0:

printi"Highest usage:", highest_name, highest_units, “units")

8. Test Cases

Hausehald Units Expected result

Asha 8 70

Balu 18 20

Chitra 36 360 + B50 penalty = 410

9. Sample Output

Number of households: 2

Household name: Asha

Water units used: 8

Asha bill = Rs. 70

Household nane: chitra

Water units used: 15

Excessive usage!

Chitra bill = Rs. 410

Highest usage: Chitra 35 units

10. Conclusion

The program calculates progressive water bills, applies penalties, and identifies excessive usage.

Python Case Studies 25 34 B.Tech Ist Yoar Page 2

CASE STUDY 26: Nested Loop Examination Seating Generator

1. Problem Statement

Create a classroom seating plan using row and column numbers. Some seats are reserved, and alternate seats must remain empty.

2. Objectives

+ Generate seats using nested loops.

+ Mark reserved seats clearly.

+ Leave altemate seats empty and show the final arrangement.

3. Assumptions

« Rows and columns are positive integers.
+ Seat numbering starts from row 1 and column 1,

+ Seats (1, 2) and (2, 3) are reserved when they exist.

+ Seats where row + column is even are left empty as an alternate-seat rule.

4. Algorithm

1. Read rows and columns.

2. Visit each position using nested loops.

3. If the seat is reserved, print R; otherwise if it is an altemate empty seat, print X; else print S.

4, Print each row on a new line.

5. Flowchart (text inside standard symbols)

_— —— -————————_

```
 START
```

INPUT rows, columns
FOR row = 1 TO rows:

```
 FOR col = 1 TO columns:
```

IE (row,col) is reserved: PRINT R

```
 ELSE IF (rowtcol) is even: PRINT X
 ELSE: PRINT S
 END FOR
```

PRINT newline

```
 END FOR
```

sToe

7. Python Source Code

rows = int (input ("Number of rows: *}+
cols = iot Uinput ("Number of columns: "))

```
 reserved = [(1, 2), (2, 31]
```

if cows <= 0 oF cols <= 0:

print ("favalid classroom size")

```
 else:
```

for x in range (1, rows + 1):

```
 for ¢ in range(1, cols + 1):
```

Sf (e, @) in reserved:

```
 print(a", end=" ")
 elif (e+ c) 4.2 == 0:
```

print ("%", end=" ")

```
 else:
```

print ("s", end=" *)

print (+

print ("S = Available, R = Reserved, X = Empty alternate seat")

8. Test Cases

Python Case Studies 26 34 B.Tech Ist Year Page 3

Rows «Columns Expected result
2x3 Rand X/S markers shown in 2 rows
0x4 Invalid classroom size

9. Sample Output

Number of rows: 2
Number of columns: 3

XRX

SKR

```
 S = Available, R = Reserved, X = Empty alternate seat
```

10. Conclusion

Nested loops help create a simple seating chart while following reserved-seat and alternate-seat rules.

Python Case Studies 25 34 B.Tech Ist Year Page 4

CASE STUDY 27: Advanced Numeric Pattern Generator

1. Problem Statement

Print a hollow diamond pattern using a user-given size. Reject invalid sizes so the output stays properly aligned.

2. Objectives

+ Use loops to print spaces and stars.

* Create a hollow diamond pattern.

+ Validate the size entered by the user.

3. Assumptions

« Size must be an integer trom 1 to 20.

+ The pattern uses stars (*) for the border.

+ For size 1, one star is displayed.

4. Algorithm

1, Read size.

2. It size is outside 1-20, display an error,

3. Print the top half and bottom half using loops.

4. Print stars only at the two border positions, except the first and last rows.

§. Flowchart (text inside standard symbols)

aw a / INPUT / . PROCESS,

```
 START
```

INPUT

IE a © 1 OR n > 20: PRINT error

```
 ELSE:
 FOR i = 1100:
```

PRINT leading spaces and hollow upper row

```
 FOR i = acl DOWN TO 1:
```

PRINT leading spaces and hollow lower row

sToP

7. Python Source Code

```
n= int (input ("Enter diamond size (1-20): "})
```

ifm < Lorn > 20:

priot ("Invalid size")

```
 elifin == 1:
```

print {7**)

```
 else:
 for i in range(n):
 spaces =n -i-1
```

if iss 0:

printit * + spaces + "**)

```
 else:
 print(" * * spaces + "rt ¢ "4 42Hi = 1) eo tEy
 for i in range(n - 2, -1, -L):
 spaces = 1 - i -1
```

if iss 0:

```
 print(" " * spaces + "*")
 else:
```

priot(" " * spaces + Tt + 7 5 (2Hi - 1) + mT

8. Test Cases

Input Expected result

1 One star

Python Case Studies 26 34 B.Tech Ist Year Page 5

Input Expected resutt
4 Hollow diamond with 7 rows
0 Invald size

9. Sample Output

Enter diamond size (1-20): 3

10. Conclusion

The program uses loops and conditions to print a hollow diamond and validates the given size.

Python Case Studies 26 34 B.Tech Ist Year Page 6

CASE STUDY 28: Repeated Transaction Detection

1. Problem Statement

Read transaction IDs and amounts, identify repeated IDs, flag transactions above a set limit, and display suspicious entries.

2. Objectives

+ Store transaction IDs and amounts.

+ Detect duplicate transaction IDs.

+ Flag amounts greater than 10,000.

3. Assumptions

« Each transaction has an ID and a non-negative amount.

+ An ID appearing more than once is treated as a duplicate.

+ Any amount above 110,000 is a threshold violation.

4. Algorithm

1. Read the number of transactions.

2. For each transaction, read ID and amount.

3. Check whether its ID was already seen and whether its amount exceeds 10,000.

4. Print a warning for each suspicious transaction.

§. Flowchart (text inside standard symbols)

aw a / INPUT / . PROCESS,

```
 START
```

INPUT

```
 seen = empty set
 FOR each transaction:
```

INPUT Id, amount

```
 suspicious = false
 IF id in seen: suspicious = true; mark duplicate
```

ADD id to seen

TF amount > 10000: suspicious = true; mark high amount

```
 IF suspicious: DISPLAY transaction as suspicious
 STOP
```

7. Python Source Code

n= int (input ("Number of transactions: "))

```
 seen = set ()
 for i in range(n}:
 tid = Laput ("Transactlon 10; ")
```

amount = float (input {"Amount: *}}

reasons - []

if cid in seen:

reasons.append ("duplicate ID")

seen. add (tid)

if amount > 10a00:

reasons. append(*amount above Rs, 16000")

if reasons:

print ("Suspicious:", tid, amount, “-", *, *. join (reasons) }

```
 else:
```

print (*Normal:", tid, amount}

8. Test Cases

Python Case Studies 26 34 B.Tech Ist Year Page 7

1D Amount Expected reautt
TO1 500 Normal

To1 700 Suspicious: duplicate 1D

TO3 19000 ‘Suspicious: high amount

9. Sample Output

Number of transactions: 3

Transaction ID: TOL

Amount: 500

Normal: TO1 $00.0

Transaction ID: TOL

Amount: 700

Suspicious: TO1 700.0 - duplicate ID

Transaction ID; 103

Amount: 15000

Suspicious: T03 15000.0 - amount above Re. 10000

10. Conclusion

The program detects repeated IOs and unusually large transactions for further checking.
Python Case Studies 25 34 B.Tech Ist Yoar Page 8

CASE STUDY 29: Password Policy Validation Engine

1. Problem Statement

Check sample passwords against basic rules: minimum length, uppercase, lowercase, digit, special character, repeated characters,

and a prohibited word,

2. Objectives

+ Check each password rule.

+ Explain which conditions fail.

+ Display whether the password passes alll rules.

3. Assumptions

*« Minimum length is 8 characters.

+ A valid password needs uppercase, lowercase, digit, and special character.

+ Three identical characters in a row are not allowed.

+ The word ‘password! is prohibited, ignoring letter case.

4. Algorithm

1, Read a password,

2. Check each rule and collect failed conditions.

3. If no conditions fail, print Valid; otherwise print Invalid and the reasons.

5. Flowchart (text inside standard symbols)

— —— -————_

START

INPUT password

errors = empty list

CHECK length, uppercase, lowercase, digit, special character

CHECK three repeated characters

CHECK prohibited word ‘password’

If errors is empty: PRINT Valid

ELSE: PRINT Invalid and errors

STOP

7. Python Source Code

```
import string
password = input ("Enter password: *}
 errors = [J
if len(password) < 8:
errors.append{"must have at least 8 characters"}
if not any(ch.isupper() for ch in password) :
errors.append{"needs an uppercase letter")
if not any(ch.islower() for ch in password):
errors.append{"needs a lowercase letter")
if not any(ch.isdigit() for ch in password):
errors.append{"aeeds a digit”)
if mot any(ch in string-punctuation for ch in password):
errors.append("needs a special character")
if any (password[i] == password[i+l) == password(i+2)
 for i in range(len(password) - 2)):
errors.append{"three repeated characters are not allowed")
if "password" in password. lower():
errors.append{*must not contain the word 'password’™)
if errors;
print {"Invalid password:"}
```

```
for error in errors:
```
print (*-", error)

Python Case Studies 26 34 B.Tech Ist Year Page 9

```
else:
```
print {"Valid password")
8. Test Cases
Password Expected reault
Abc@1234 Valid password
abe Invalid: multiple rules tail
Password123! Invalid: prohibited word
9. Sample Output
Enter password: abc
Invalid password:
= must have at least chaxacters
needs an uppercase letter
- needs a digit
~ needs a special character
10. Conclusion
The program checks several password rules and clearly explains any failed conditions.
Python Case Studies 26 34 B.Tech Ist Year Page 10

CASE STUDY 30: Student Email and Identifier Validation

1. Problem Statement

Validate student email addresses and registration [Os using simple formatting rules and report invalid entries.

2. Objectives

+ Check email structure

* Check registration ID structure.

+ Explain invalid formatting.

3. Assumptions

+ Email must contain one @, a dot after @, and no spaces.

+ Registration ID format is two uppercase letters followed by four digits, such as CS2026.

+ This is a basic classroom validator, not a complete internet email validator.

4. Algorithm

1. Read an email and registration ID.

2. Check the email rules.

3. Check the registration ID using length and character positions.

4. Display valid/invalid status and reasons.

§. Flowchart (text inside standard symbols)

aw a / INPUT / . PROCESS,

```
 START
```

INPUT email, reg_id

If email has one @, dot after @, and ao spaces: email valid

```
 ELSE: email invalid
```

IE veg_id has 2 uppercase letters + 4 digits: ID valid

```
 ELSE: ID invalid
 DISPLAY results
 STOP
```

7. Python Source Code

email = input("Enter student email: "
reg id = input ("Enter registration ID: ")
if (email.count ("@") == 1 and * " not in email

and *.7 in email.split("@*) [-2]

and email.split¢"@") [-1].find«(".") > 0s

print {"Email: Valid")

```
 else:
```

print ("Email: Invalid format")
if (len{reg_id} == 6 and reg_id{:2].isalpnacy

and reg_id[:2].isupper() and reg_id[2:] isdigit 0}+

print Registration ID: valid")

```
 else:
```

print «"Registeation 10: Invalid format")

8. Test Cases

Email /10 Expected reautt

student@college in /CS2026 Both valid

student college.in/cs20A6 Both invalid

9. Sample Output

Enter student email: student @college.in
Enter registration ID: CS2026

Python Case Studies 26 34 B.Tech Ist Year Page 11

Email: Valid
Registration ID: Valid

10. Conclusion

The program applies simple string checks to identify correctly formatted student emails and (Ds.

Python Case Studies 25 34 B.Tech Ist Yoar Page 12

CASE STUDY 31: String-Based Examination Answer Evaluation

1, Problem Statement

Compare a student's answer string with an answer key, count correct, incorrect, and unanswered questions, apply negative marking,

and show the score.

2. Objectives

+ Compare each answer with the key.

+ Count correct, wrong, and blank answers.

+ Calculate the final score using negative marking.

3. Assumptions

+ Answers are entered as letters separated by spaces.

* Correct answer = +1 mark; wrong answer = -0.25 mark; blank answer = 0.

+ The answer key and student answers have equal length.

4. Algorithm

1. Read the answer key and student answers.

2. Split both into lists.

3. Compare answers position by position.

4. Count correct, wrong, and blank answers, then calculate score.

5. Flowchart (text inside standard symbols)

— —— -————_

```
 START
```

INPUT key_string, answer_string

```
 key = split (key_string); answers = split (answer_string)
```

SET correct=0, wrong=0, blank=0

```
 FOR each position:
 IF answer is blank: blank += 1
 ELSE IF answer equals key: correct += 1
```

BLSE: wrong ++ 1

```
 score = correct - wrong*0.25
 DISPLAY counts and score
 STOP
```

7. Python Source Code

key = input ("Answer key (use spaces}: *).upper{) split ()
angwere = input ("Student answers (use spaces; use - for black): “}.upper().aplit (}

if len(key) != len (answers) :

print {"Error: answer counts do not match")

```
 else:
 correct = wrong = blank = 0
```

tor ky 8 in zip(key, answers):

if ase tm

blank $= 1

```
 elif a == k:
```

correct ¢= 1

```
 else:
```

wrong += 1

```
 score = correct - wrong * 0.25
```

priot ("Correct :*, correct)

prant{*Incorreet:", wrong?

print {"Unanswezed:", blank)

priot ("Pinal score:", score}

8. Test Cases

Python Case Studies 25 34. B.Tech Ist Year Page 13

Key /Answars Expected result

ABCDI/AB-A Zcorect. 1 wrong, 1 blank, score 1.75

ABIAB 2 correct, score 2.0

9. Sample Output

Answer key (use spaces}: ABCD

Student answers (use spaces; use for blank): AB A

Correct: 2

Incorrect: 1

Unanswered: 1

Final score: 1.75

10. Conclusion

The program evaluates answers, counts each response type, and calculates a score with negative marking.

Python Case Studies 26 34 B.Tech Ist Year Page 14

CASE STUDY 32: Text-Based Invoice Data Extraction

1, Problem Statement

Read invoice records written in a simple text format and extract product code, quantity, and amount. Flag records that do not follow
the format.

2. Objectives

* Split invoice text into fields.

+ Extract code, quantity, and amount.

+ Identify malformed records.

3. Assumptions

+ Record format is CODE,QUANTITY,AMOUNT, for example P101,2,150.50.
* Quantity must be a positive integer and amount must be non-negative.

+ Fields are separated by commas.

4. Algorithm

1. Read a record.

2. Split it using commas.

3. Check that there are exactly three fields.

4. Convert quantity and amount and print the values; report an error if conversion or validation fails.

5. Flowchart (text inside standard symbols)

— —— -————_

START
INPUT record
fields = split record by comma
If number of fields is not 3: PRINT malformed

ELSE:

TRY convert quantity and amount

```
IF quantity <= 0 OR amount < 0: PRINT malformed
ELSE; DISPLAY code, quantity, amount
IF convereion fails; PRINT malformed
STOP
```

7. Python Source Code

record = input ("Enter invoice record (CODE,QUANTITY, AMOUNT}: ")
fields = record. split (™,")
if len(fields) != 3:

print {"Malformed iovolce record")

```
else:
code = fields [4] stripy)
```

try:

quantity - int (fields{1].stripu)

```
amount = float (fields [2] .stripi))
```

i€ code == "* or quantity <= 0 or amount < 0:

print(*"Malformed invoice record"

```
else:
```

print ("Product code:", code}

priot ("Quantity:", quantity)

print j*Amount: Rs.", amount}

except ValueError:

Print ("Malformed invoice record: quantity/amount is not numerict)
8. Test Cases

Record Expected result
P101,2,150.80 Valid; code P101, quantity 2, amount 150.50

Python Case Studies 2 34 B.Tech Ist Year Page 15

Recard Expoctod result

P102,tw0.80 — Malformed: quantity is nol numeric

103,020. Malformed: quantity must be positive

9. Sample Output

Enter invoice record (CODE, QUANTITY, AMOUNT): P101,2,150.50
Product code: P101

Quantity: 2

Amount: Rs. 150.5

10. Conclusion

The program uses basic string operations and exception handling to extract invoice details safely.

Python Case Studies 26 34 B.Tech Ist Year Page 16

CASE STUDY 33: Campus Registration Identifier Analyzer

1. Problem Statement

Check student registration IDs, extract their branch and admission year, and classify valid records.

2. Objectives

+ Validate the identifier format.

+ Extract branch and admission year.

Display the classification for each ID.

3. Assumptions

+ Format is BRANCH + YEAR + NUMBER, e.g. CSE2026123.

+ Branch is three uppercase letters, year is four digits, and final number is three digits.

+ The total ID length is 10 characters.

4. Algorithm

1. Read a registration ID.

2. Check its length and character groups.

3. If valid, extract branch, year, and number,

4. Display the parts; otherwise display Invalid ID.

§. Flowchart (text inside standard symbols)

aw a / INPUT / . PROCESS,

```
 START
```

INPUT reg_id

IE length is 10 AND first 3 are uppercase letters

AND next 4 are digits AND last 3 are digits:

```
 branch = first 3 characters
 year = next 4 characters
 number = last 3 characters
 DISPLAY branch, year, number
```

RLS: DISPLAY Invalid 1D

```
 STOP
```

7. Python Source Code

```
reg_id = input ("Enter registration ID: *)
if (len(reg_d) == 10 and reg_id(:3].isalpha()
and reg_id(:3),isupper{) and reg_id[3:7].isdigit (
and reg_id(7:10} .isdigit (9):
 branch = reg_id(:3]
 year = reg_id(3:7)
 number = reg_ia(7:10]
printi*"Branch:", branch)
print ("Admission year:", year)
printi"Student number:", number}
 else:
printi"invalid registration 1D")
```

8. Test Cases

Input Expected reautt

CSE2026123 Valid: CSE, 2026, 123

652026123 Invalid: wrong length’branch case

CSE20A6123 Invalid: year contains a letter

9. Sample Output

Python Case Studies 26 34 B.Tech Ist Year Page 17

Enter registration ID: CSE2026123
Branch: SSE
Admission year: 2026
Student umber: 123
10. Conclusion
The program separates a valid campus !D into branch, admission year, and student number.
Python Gase Studies 25 34 B.Tech Ist Year Page 18

CASE STUDY 34: Student Feedback Sentiment Scoring

1. Problem Statement

Analyse feedback sentences using positive and negative keywords, calculate a simple score, and classify each sentence.

2. Objectives

+ Count positive and negative words.

* Calculate a sentiment score.

+ Classify feedback as Positive, Negative, or Neutral.

3. Assumptions

« Positive words: good, helpful, excellent, clear.

+ Negative words: bad, confusing, poor, slow.

+ Score = positive count - negative count. Score above 0 is Positive, below 0 is Negative, and 0 is Neutral.

4. Algorithm

1, Read a feedback sentence and convert it to lowercase.

2. Count positive and negative keywords found in the sentence.

3. Calculate the score.

4. Classify the feedback based on the score and display the result.

§. Flowchart (text inside standard symbols)

aw a / INPUT / . PROCESS

```
START
```

INPUT feedback

```
text = lowercase (feedback)
positive_count = count positive keywords
hegative_count = count negative keywords
score = positive_count - negative_count
```

IE score > : sentiment = Positive

```
ELSE IF score < 0: sentiment = Negative
```

RISE: sentiment ~ Neutral

```
DISPLAY counts, score, sentiment
STOP
```

7. Python Source Code

feedback = input("Enter student teedback: “}.lower(}

Positive words = ["good", "helpful", "excellent", "clear"]

```
oegative_words = ["bad", "confusing", "poor*, "slow*]
posltive_count = 0
negative_count = 0
for word in positive_words:
```

if word in feedback:

positive_count += 1

```
for word in negative_worda:
```

if word in feedback:

negative_count += 1

```
score = positive_count - negative_count
```

if score > a:

```
sentiment = "Positive"
elif score < D:
sentiment = "Neqative”
else:
sentiment = "Neutral”™
```

print ("Positive keywords:", positive_count)

print (*Negative keyworde:", negative count)

Python Case Studies 26 34 B.Tech Ist Year Page 19

print ("Score:", score?

print (“Sentiment:", sentiment}

8. Test Cases

Feedback Expected reautt

The class was good and clear Positive (score 2)

‘The lesson was confusing and slow Negative {score -2)

The class starts at ten Neutral (score 0)

9. Sample Output

Enter student feedback; The class was good and clear

Positive keywords: 2

Negative keywords: 0

Score: 2

Sentiment: Positive

10. Conclusion

The program demonstrates a beginner-friendly keyword methad for scoring and classifying student feedback.

Python Case Studies 25 34 B.Tech Ist Yoar Page 20